YAPAY ZEKÂ

ORTAOKUL

DENEYAP TÜRKİYE

#MİLLİ TEKNOLOJİ HAMLESİ

İSTANBUL DENEYAP TEKNOLOJİ ATÖLYESİ

YAPAY ZEKA DERSİ

7. Hafta: Zeki Optimizasyon

Intelligent Optimization

T.C. SANAYİ VE TEKNOLOJİ BAKANLIĞI | T.C. GENÇLİK VE SPOR BAKANLIĞI | TÜBİTAK | TÜRKİYE TEKNOLOJİ TAKIMI

T3 DENEYAP EĞİTMENİ (Hazırlayan ve Sunan): Muhammed Emin Korkunç

(Bilgisayar Mühendisi)

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.ALGILA

- 1.1. Optimizasyon Kavramı
- 1.2. Zeki Optimizasyon Kavramı
- 1.3. Genetik Algoritma ile Zeki Optimizasyon
- 1.4. Karınca Koloni Optimizasyonu ile Zeki Optimizasyon

2. TASARLA

3. HAREKETE GEÇ

4. YÜRÜT

5. KARAR VER

- 5.1. Sonuçların Gözlemlenmesi

6. İLAVE ETKİNLİK

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

7. Hafta: Zeki Optimizasyon

- Küçük Bir Soru: Bir yere giderken en kısa yolu nasıl seçersiniz? Ya da çantanıza en çok eşyayı en düzenli nasıl sığdırırsınız?
- İşte Cevap: Hayatımızın her anında aslında birer "Optimizasyon Uzmanı" gibi davranıyoruz!
- Bu Hafta Neler Yapacağız?
  - Doğadan ilham alan süper algoritmaları öğreneceğiz.
  - Karıncaların nasıl bu kadar akıllıca yol bulduğunu göreceğiz.
  - Kendi Python kodlarımızla birer "Zeki Çözüm" üreteceğiz!

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

3- Optimizasyon Nedir? (En Verimli Yol)

"Optimizasyon" da Ne Demek?

- Sözlük Anlamı: Elindeki imkanlarla en iyi, en hızlı veya en ucuz sonucu alma çabasıdır.
- Örnek: Okula giderken trafiğe takılmamak için seçtiğin yol.
  - Bir bilgisayar oyununda en az can kaybederek bölümü geçme stratejin.
- Zeki Optimizasyon: Bilgisayarların tıpkı insanlar veya hayvanlar gibi düşünerek karmaşık problemleri çözmesidir.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

4- Bugünün Süper Güçleri: Algoritmalar

Neleri Öğreneceğiz? (Kazanımlarımız)

Bugün cebimize şu süper güçleri koyacağız:

1. **Genetik Algoritmalar: Canlıların gelişiminden ilham alarak en iyi çözümü "üreteceğiz".**
2. **Karınca Kolonisi: Minicik karıncaların yemek bulma taktiğini bilgisayara öğreteceğiz.**
3. **Python ile Kodlama: Bu zekayı kendi ellerimizle koda dökeceğiz!**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

5- Akış Planımız: Nasıl İlerleyeceğiz?

Bugünkü Yol Haritamız

- 👀 Algıla: Optimizasyon dünyasına giriş yapıyoruz.
- 🎨 Tasarla: Çözüm yollarımızı kodlar üzerinden örneklerle planlıyoruz.
- 🚀 Harekete Geç: Python editörlerimizi açıyoruz ve kod yazmaya başlıyoruz!
- 🤖 Yürüt & Karar Ver: Yazdığımız kodlar nasıl çalışıyor? Karıncalarımız doğru yolu buldu mu? Hep beraber test ediyoruz.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

6- "En İyileme" Sanatı: Optimizasyon

Nedir Bu Optimizasyon?

- **Diğer Adı: En iyileme.**
- **Sırrı: Elimizdeki seçenekler arasından "en uygun" olanı seçme işidir.**
- **Günlük Hayattan: Aslında her gün kafandan plan yaparken birer matematikçi gibi çalışıyorsun!**
  - ***Örnek:*** **"En kısa yoldan bakkala nasıl giderim?" dediğinde bir yol optimizasyonu yapıyorsun.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

7-Haydi Hesaplayalım! (Matematiksel Model)

Geçmek İçin Kaç Almalısın?

Bir dersten geçme puanını hesaplayalım. Formülümüz şu olsun:
Başarı Puanı = (Yazılı x 0.6) + (Sözlü x 0.4)

- **Senaryo: Yazılıdan 50 aldın. Dersten geçmek için en az 60 puan lazım.**
- **Soru: Sözlüden en az kaç almalısın?**
- **Cevap: Sözlüden ? alırsan tam 60 puanla geçersin. 80 alırsan daha da iyi olur!**
- **İşte Optimizasyon: Hedefine ulaşmak için "bilinmeyen" (Sözlü notu) değerini buldun!**

Kavramlar: > * Değişken: Puanların (y ve s)

- **Katsayı: Notun ağırlığı (0.6 ve 0.4)**
- **Sabit: Eğer öğretmen herkese +10 puan verirse bu sabittir.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

8- Gerçek Dünyada Optimizasyon

Mühendisler Bunu Nerede Kullanır?

Sadece not hesaplamak için değil, dünyayı kurtarmak (veya verimli kılmak) için de kullanıyoruz:

1. **Sıfır Atık: Kumaşı öyle bir keselim ki hiç parça artmasın.**
2. **Güneş Avcıları: Bir odanın en çok ışık alması için pencereyi tam nereye koymalıyız?**
3. **Kurye Rotası: Bir kargo aracı 10 eve en az yakıtla nasıl uğrar?**
4. **Gizli Karışım: En dayanıklı betonu yapmak için çimento, su ve kumu hangi oranda karıştırmalıyız?**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Zeki Optimizasyon (Yapay Zekanın Arama Gücü)

- Klasik Yöntemler Yetmeyince: Bazen problemler o kadar karmaşıktır ki, eski matematik yöntemleri tıkanıp kalır.
- Doğadan İlham: Bilgisayarlar; kuş sürülerinin, karınca kolonilerinin ve hatta nehirlerin akışının "en iyiyi bulma" taktiklerini taklit eder.
- Öğrenmek Yok, Aramak Var: Bu sistemler bir şey "ezberlemez". Bunun yerine, döngüler kullanarak doğru cevabı bulana kadar deneme yaparlar.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

10- "N" Sayıda Kahramanın Görevi

Bir Elin Nesi Var, N Tane Etmenin Sesi Var!

Zeki optimizasyonda tek bir kişi değil, koca bir ekip çalışır:

- **Sayıca Çokluk (N): Probleme aynı anda saldıran çok sayıda "etmen" (robot, karınca veya parçacık) vardır.**
- **Rastgele Başlangıç: Herkes işe farklı ve rastgele bir noktadan başlar.**
- **Yardımlaşma: Bir etmen güzel bir sonuç bulduğunda, diğerlerine "Hey! Ben burada iyi bir şey buldum!" der ve diğerleri de ona doğru yaklaşır.**

Bilgisayara bir problem çözdürürken ona sınırları söylemeliyiz:

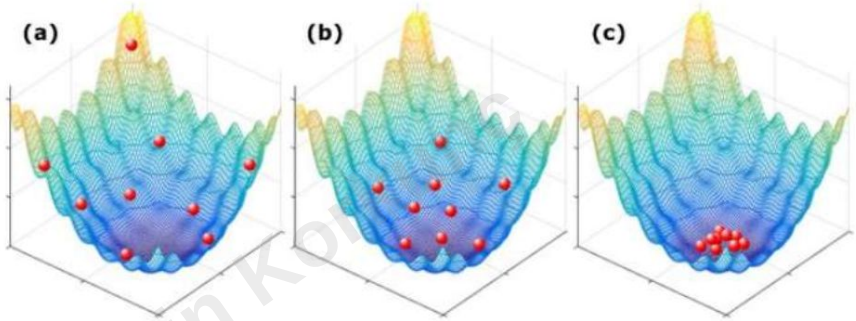
- **Sınır Değerleri: Notlar 0'dan küçük, 100'den büyük olamaz!**
- **Denge Kurmak: Bir şirket hem Kârını artırmak hem de Zararını azaltmak ister. Bazen ikisini birden dengeleyen "ortadaki en iyi" değeri bulmak gerekir.**
- **Sürekli vs. Kombinasyonel:**
  - ***Sürekli:*** **"Hangi derecede ısıtmalıyım?" (Sayı bulma)**
  - ***Kombinasyonel:*** **"Hangi şehirleri hangi sırayla gezmeliyim?" (Sıralama bulma)**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.2. Zeki Optimizasyon Kavramı



Şekil 7.1. Görsel olarak tipik bir optimizasyon süreci (Web Kaynağı 7.1)

Hazırladığın görsele bakalım; kırmızı noktalar bizim çözüm arayan robotlarımız olsun:

- **(a) Herkes Dağınık: İlk başta robotlar her yere dağılmış durumda, kimin nerede olduğu belli değil.**
- **(b) Sinyali Alan Yaklaşıyor: Birkaç robot daha derin bir yer bulduğunda, diğerleri de o bölgeye doğru toplanmaya başlıyor.**
- **(c) Hedefe Ulaşıldı: Sonunda tüm ekip, aradığımız "en derin" (en uygun) noktada birleşiyor.**

Dikkat: Bazen robotlar yanlış bir çukura düşebilir (yanlış tespitler). Zeki optimizasyon %100 garanti vermez ama her zaman çok etkili sonuçlar üretir!

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Doğadaki Gizli Matematikçiler

Algoritmalarımızı tasarlarken şu kahramanlara isimler veriyoruz:

- 🐝 **Arılar: En çok polen olan çiçeği nasıl buluyorlar?**
- 🐜 **Karıncalar: En kısa yolu nasıl işaretliyorlar?**
- 🌊 **Akarsular: Denize ulaşmak için engelleri nasıl aşıyorlar?**
- 🌸 **Çiçekler: Tozlaşma için en uygun zamanı nasıl seçiyorlar?**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

13- Genetik Algoritma (Doğanın Akıllı Kodu)

En Güçlüler Hayatta Kalır!

- **Süper Gücü: Güçlü ve başarılı çözümlerin özelliklerini yeni nesillere aktararak başarı şansını artırır.**
- **Nasıl Çalışır?: En iyi sonuç veren çözümleri (ebeveynleri) birbirleriyle eşleştirir ve daha başarılı "çocuk" çözümler üretir.**
- **Evrimsel Zekâ: Başarılı çözümleri geleceğe taşıdığı için bu algoritmaya "Evrimsel Algoritma" denir.**

Düşün, Tartış: Kuşlar veya arılar yemek bulmak için nasıl iş birliği yapar?

- **Cevap: Karıncalar en kısa yolu bulup işaretler, arılar ise birbirlerine dans ederek en çok polen olan çiçeğin yerini söyler. Bilgisayarlar da bu yardımlaşmayı taklit eder.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Genler, Kromozomlar ve Kodlama

Bilgisayar bir problemi çözerken tıpkı bir canlı gibi davranır:

- **Kodlama: Problemin bilinmeyenlerini bilgisayarın anlayacağı sayılara dönüştürme işidir.**
- **Gen: Problemdeki her bir bilinmeyen değerdir.**
- **Kromozom (Birey): Genlerin birleşmesiyle oluşan her bir çözüm adayıdır.**
- **Popülasyon: Tüm çözüm adaylarının (bireylerin) oluşturduğu gruptur.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

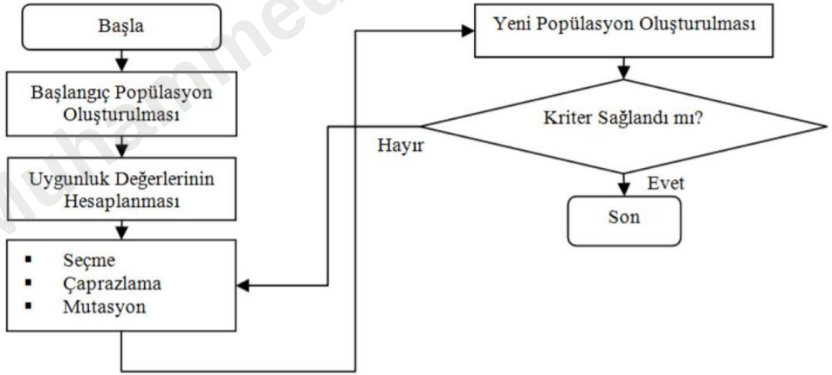
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

15- Yeni Nesil Nasıl Oluşur? (3 Temel Adım)

Seç, Eşleştir ve Değiştir!

Algoritma her döngüde şu üç işi yapar:

1. **Uygunluk Ölçümü: Her bireyin problemi ne kadar iyi çözdüğüne bakılır (Puan verilir).**
2. **Çaprazlama (Crossover): En yüksek puanlı iki "ebeveyn" seçilir ve genleri karıştırılarak "çocuklar" oluşturulur.**
3. **Mutasyon: Bazen genleri rastgele değiştirilir ve mutasyona uğrar.**



Şekil 7.2. Genetik Algoritma'nın genel akışı (Web Kaynağı 7.2)

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

16- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Genetik Algoritma neden "Evrimsel Algoritma" grubundadır?

- **Cevap: Çünkü canlılardaki nesil aktarımı sürecini taklit ederek, her nesilde daha güçlü çözümler üretir.**

Soru 2: "Mutasyon" neden gereklidir? Hep en iyileri eşleştirsek yetmez mi?

- **Açıklama/Cevap: Sadece en iyileri eşleştirirsek bir süre sonra hepsi birbirine benzer ve yeni yollar aramayı bırakırlar. Mutasyon, sisteme "sürpriz" bir yenilik katarak takılıp kalmamızı önler.**

Soru 3: Kodlama süreci nedir?

- **Cevap: Problemin bilinmeyenlerini (değişkenlerini) bilgisayarın anlayacağı genler ve kromozomlar haline getirme sürecidir.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

17- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Neden sadece en iyi çözümleri seçmiyoruz da onları bir de "çaprazlıyoruz"?

- **Cevap: Çünkü farklı iyi çözümleri birleştirerek, her ikisinin de en iyi yanlarını taşıyan "süper bir çözüm" elde etmeye çalışıyoruz.**

Soru 2: "Mutasyon" olmasaydı ne olurdu?

- **Açıklama/Cevap: Eğer mutasyon olmazsa, çözümler bir süre sonra hep birbirinin aynısı olurdu. Mutasyon, bilgisayara "Hadi farklı bir şey dene!" diyerek yeni çözümler bulmasını sağlar.**

Soru 3: Genetik algoritma bir şeyi "ezberler mi"?

- **Cevap: Hayır! Genetik algoritma öğrenmez; döngüler boyunca deneme-yanılma ve geliştirme yaparak en iyiyi arar.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Minik Karıncaların Büyük Zekası

- İlham Kaynağı: Karıncaların yiyecek bulurken birbirlerine yol gösterme yöntemi.
- Gizli Silah: Feromon! Karıncalar yürüdükleri yollara "feromon" adı verilen kimyasal bir koku bırakırlar.
- En Kısa Yolun Sırrı: Kısa yolu kullanan karıncalar daha hızlı gidip geldikleri için o yoldaki koku daha güçlü olur. Diğer karıncalar da bu güçlü kokuyu takip eder.
- Kullanım Alanı: Bu algoritma, özellikle "Kombinasyonel Optimizasyon" (yani en iyi sıralamayı bulma) işlerinde bir numaradır.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

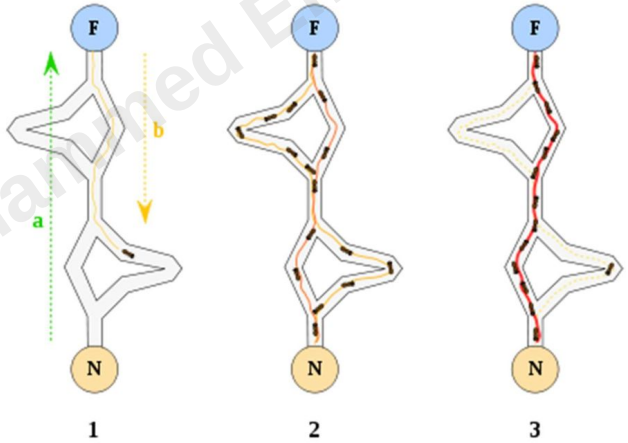
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

19- Şekil 7.3: Adım Adım En Kısa Yol

N'den F'ye En Akıllı Rota

Hazırladığın görsele (Şekil 7.3) birlikte bakalım:

1. **Aşama 1: Karıncalar yola ilk çıktığında her yöne dağılırlar, henüz koku çok zayıftır.**
2. **Aşama 2: Bazı karıncalar tesadüfen daha kısa yollar keşfeder ve oraya daha çok koku bırakmaya başlarlar.**
3. **Aşama 3 (Final): Koku (feromon) değeri en yüksek olan yol kıpkırmızı olur! Artık bütün koloni en kısa ve en uygun yolu bulmuştur.**



Şekil 7.3: Karınca Kolonisi Optimizasyonu ile temsili problem çözümü (Web Kaynağı 7.3)

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

20- Algoritma Nasıl "Karar" Verir?

Bilgisayar Karıncaları İş Başında!

Algoritmanın içindeki mekanizma şöyledir:

- N Sayıda Karınca: Sistemde çok sayıda sanal karınca tanımlanır.
- Koku Güncelleme: Başarılı bir rota çizen karıncanın geçtiği yolların puanı (feromon değeri) artırılır.
- Olasılık Hesabı: Bir karınca yol ayrımına geldiğinde, kokusu daha yoğun olan tarafı seçme ihtimali daha yüksektir.
- Sonuç: Kod durduğunda, en çok "puan" toplayan yol bizim en iyi çözümümüz olur.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

21- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Karınca Kolonisi ve Genetik Algoritma arasındaki fark nedir?

- **Cevap: Genetik Algoritma daha çok "en iyi sayıları" bulmaya çalışırken (Sürekli), Karınca Kolonisi "en iyi sıralamayı veya yolu" bulmaya (Kombinasyonel) odaklanır.**

Soru 2: Feromon (koku) zamanla uçup gitse ne olur?

- **Açıklama/Cevap: Aslında bu iyi bir şeydir! Eğer koku hiç uçmasaydı, karıncalar eski ve kötü yollara takılıp kalabilirdi. Kokunun uçması, sistemin her zaman "daha taze ve daha iyi" yollar aramasına izin verir.**

Soru 3: Başka hangi hayvanların algoritmaları olabilir?

- **Cevap: Ateş böcekleri, guguk kuşları ve hatta su damlaları bile bu iş için ilham kaynağıdır!**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

22- Simülasyon ile Zeki Optimizasyon





Simülasyon ile Zeki Optimizasyon

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

2. TASARLA: Bir Matematik Bilmecesi

- Görevimiz: Öyle 6 tane sayı $x1, x2, x3, x4, x5, x6$ bulalım ki, aşağıdaki işlemin sonucu tam 44 çıksın!
- Denklemimiz: $f(x1, x2, x3, x4, x5, x6) = 4x1 - 2x2 + 3{,}5x3 + 5x4 - 11x5 - 4{,}7x6$

- Yardımcımız: PyGAD isimli özel bir **Python** kütüphanesi.
- pip install pygad = kütüphane kurulumu

Not: "Bu denklemi kafadan çözmek çok zor ama Genetik Algoritma bunu bizim için saniyeler içinde yapacak!"

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1. Kütüphanelerin Tanımlanması

H7_genetik-algoritma.py

```python
import pygad
import numpy
```

- `pygad`: Genetik algoritmaları kolayca oluşturmak, eğitmek ve optimize etmek için kullanılan temel kütüphanedir.
- `numpy`: Matematiksel hesaplamalar ve dizi (array) işlemleri için kullanılır. Burada gen katsayılarını çarpmak için tercih edilmiştir.

2. Girdiler ve Hedefin Belirlenmesi

```python
function_inputs = [4,-2,3.5,5,-11,-4.7]
desired_output = 44

def fitness_func(ga_instance, solution, solution_idx):
    output = numpy.sum(solution * function_inputs)
    fitness = 1.0 / numpy.abs(output - desired_output)
    return fitness
```

`function_inputs`: Denklemdeki sabit katsayıları temsil eder. Algoritma bu sayıları değiştiremez.
`desired_output`: Ulaşmaya çalıştığımız sonuçtur. Algoritma, girdilerle çarpıldığında 44 sonucunu veren en uygun 6 sayıyı (geni) arar.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

3. HAREKETE GEÇ: Kim Daha Yakın?

Bilgisayar hangi çözümün daha iyi olduğunu nasıl anlıyor?

- **Fitness (Uygunluk): Bulunan sonucun 44'e ne kadar yakın olduğunu ölçer.**
- **Hedef: Sonuç 44'e yaklaştıkça uygunluk puanı artar.**
- **Özet: `nesil_ozeti` fonksiyonu ile her adımda bilgisayarın ne kadar ilerlediğini ekranda göreceğiz.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

3. Uygunluk (Fitness) Fonksiyonu

```
6
7   def fitness_func(ga_instance, solution, solution_idx):
8       output = numpy.sum(solution * function_inputs)
9       fitness = 1.0 / numpy.abs(output - desired_output)
10      return fitness
```

solution: O anki nesilde denenen katsayı adayıdır.

output: Girdiler ile çözüm adayının çarpım toplamıdır (Y = \sum w_i x_i).

fitness: Bir çözümün ne kadar "kaliteli" olduğunu belirler. Hata payı (fark) azaldıkça fitness değeri artar.

Not: Paydaya eklenen 0.000001, farkın sıfır çıkması durumunda "sıfıra bölünme" hatasını (ZeroDivisionError) engellemek içindir.

4. Algoritma Parametreleri

```
14      num_generations = 100
15      num_parents_mating = 7
16
17      sol_per_pop = 50
18      num_genes = len(function_inputs)
19
```

`num_generations = 100` (Nesil Sayısı): Algoritmanın kaç tur (döngü) boyunca evrimleşeceğini belirler.

`num_parents_mating = 7` (Ebeveyn Sayısı): Yeni nesli üretmek için seçilecek en iyi "anne-baba" çözüm sayısıdır.

`sol_per_pop = 50` (Popülasyon Sayısı): Her nesilde aynı anda yarışan aday çözüm sayısıdır.

`num_genes = 6` (Gen Sayısı): Aranan bilinmeyen katsayı sayısıdır (Burada 6 adet).

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

5. İzleme (Callback) Fonksiyonu

```
last_fitness = 0
def nesil_ozeti(ga_instance):
    global last_fitness
    print("Nesil = {generation}".format(generation=ga_instance.generations_completed))
    print("Fonksiyon Sonucu = {fitness}".format(fitness=ga_instance.best_solution()[1]))
    print("Degisim = {change}".format(change=ga_instance.best_solution()[1] - last_fitness))
    last_fitness = ga_instance.best_solution()[1]
```

- `last_fitness = 0`: Gelişimi ölçmek için başlangıç puanını sıfırlar.
- `def nesil_ozeti(ga_instance):`: Her nesil sonunda çalışacak takip fonksiyonunu tanımlar.
- `global last_fitness`: Fonksiyon dışındaki puan değişkenini güncelleyebilmek için çağırır.
- `print("Nesil = ...")`: Kaçıncı döngüde (nesilde) olduğumuzu yazdırır.
- `print("Fonksiyon Sonucu = ...")`: O nesildeki en iyi çözümün başarı puanını gösterir.
- `print("Degisim = ...")`: Yeni neslin eskisinden ne kadar daha iyi olduğunu hesaplar.
- `last_fitness = ...`: Bir sonraki kıyaslama için mevcut puanı kaydeder.

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

6. GA Nesnesini Başlatma ve Çalıştırma

```
ga_instance = pygad.GA(num_generations=num_generations,
                       num_parents_mating=num_parents_mating,
                       fitness_func=fitness_function,
                       sol_per_pop=sol_per_pop,
                       num_genes=num_genes,
                       on_generation=nesil_ozeti)

ga_instance.run()
```

`ga_instance = pygad.GA(`: Genetik Algoritma (GA) motorunu tüm ayarlarıyla oluşturur.
`num_generations=...`: Toplam kaç nesil boyunca deneme yapılacağını motora bildirir.
`num_parents_mating=...`: Yeni nesilleri üretecek "şampiyon" ebeveyn sayısını belirler.
`fitness_func=...`: Çözümlerin başarısını ölçmek için kullanılacak formülü tanıtır.
`sol_per_pop=...`: Her adımda kaç farklı adayın test edileceğini ayarlar.
`num_genes=...`: Aranan bilinmeyen katsayı miktarını (boyutunu) söyler.
`on_generation=...`: Her nesil bittiğinde raporlama yapacak fonksiyonu bağlar.
`ga_instance.run()`: Algoritmayı başlatır; evrimsel döngüyü (seçilim, çaprazlama, mutasyon) çalıştırır.

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

7. En İyi Çözümü Çıktılama

```
37    solution, solution_fitness, solution_idx = ga_instance.best_solution()
38    print("En uygun cozum degisken degerleri : {solution}".format(solution=solution))
39    print("En uygun cozumu veren birey indeks no.: {solution_idx}".format(solution_idx=solution_idx))
40
41    prediction = numpy.sum(numpy.array(function_inputs)*solution)
42    print("En uygun cozum ile fonksiyon sonucu : {prediction}".format(prediction=prediction))
43
44    if ga_instance.best_solution_generation != -1:
45        print("En uygun cozum {best_solution_generation} nesil sonra elde edildi."
46              .format(best_solution_generation=ga_instance.best_solution_generation))
```

- `solution, ..., ... = ga_instance.best_solution()`: 100 nesil sonunda ulaşılan en iyi sonucu (katsayıları) ve puanını hafızadan çeker.
- `print("En uygun cozum...")`: Bulunan ideal katsayıları (genleri) ekrana yazdırır.
- `print("En uygun cozumu veren birey...")`: Bu çözümün popülasyon içindeki sıra numarasını (indeksini) gösterir.
- `prediction = numpy.sum(...)`: Bulunan katsayıları gerçek formülde yerine koyarak sağlamasını yapar.
- `print("En uygun cozum ile...")`: Algoritmanın bulduğu sonucun hedefimiz olan 44'e ne kadar yaklaştığını yazdırır.
- `if ga_instance.best_solution_generation != -1:`: "Eğer bir çözüm bulunduysa" kontrolü yapar.
- `print("...nesil sonra elde edildi.")`: En iyi sonuca kaçıncı nesilde ulaşıldığını (örneğin 45. nesil) raporlar.

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

30- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Kodda `num_generations` (Nesil Sayısı) değerini 10 yaparsak ne olur?

- **Cevap: Algoritma sadece 10 kez deneme yapar. Eğer problem zorsa, 10 denemede doğru sonuca ulaşamayabilir ve yarım kalmış bir cevap verebilir.**

Soru 2: `fitness = 1.0 / numpy.abs(output - desired_output)` satırı ne işe yarar?

- **Açıklama/Cevap: Bu bir "puanlama" formülüdür. Çıkan sonuç 44'e (desired_output) ne kadar yakınsa, fark o kadar küçük olur. Paydadaki sayı küçüldükçe toplam puan (fitness) büyür. Yani 44'e en yakın olan en yüksek puanı alır!**

Soru 3: Bilgisayar bu sayıları bulurken "matematik kurallarını" mı biliyor?

```
En uygun cozum degisken degerleri : [-1.16498213  1.98641117 -2.33159974  3.01034528
-3.70851693 -1.05251707]
En uygun cozumu veren birey indeks no.: 0
En uygun cozum ile fonksiyon sonucu : 43.99889295825503
En uygun cozum 27 nesil sonra elde edildi.
```

ır, en iyileri

güyü tekrar

Şekil 7.9. Optimizasyon süreci sonucu çözüm

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

31- Yapay Zekâ Dünyayı Kurtarabilir mi?

Geleceği Tahmin Etmek: Dünya’nın Dijital İkizi

- Büyük Hedef: Bilim insanları, Dünya’nın tıpkı bir bilgisayar oyunu gibi **"Dijital İkizi"**ni (sayısal bir kopyasını) oluşturmak istiyor.

- Neden Önemli?: Gerçek dünyada yapamadığımız tehlikeli deneyleri bu laboratuvarda yapabilir, iklim değişikliğinin sonuçlarını önceden görebiliriz.

- Ulaşılamayan Yerler: Kutup bölgeleri gibi gidilmesi zor yerleri dijital ikizler sayesinde takip edebilir, ölçüm araçlarımızı (İHA ve denizaltılar) daha doğru yerlere yönlendirebiliriz.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

32- Her Şey Kusursuz mu? (Zorluklar)

Yapay Zekânın "Bilgi" Karın Ağrısı

Yapay zekâ çok güçlü ama her şeyi hemen çözemiyor:

- **Veri Eksikliği: Dr. Judah Cohen'e göre, 1979'dan beri uydu verisi toplasak da, yapay zekânın en iyi (optimum) sonucu vermesi için elimizde hâlâ yeterli veri yok.**

- **Gerçek mi, Yapay mı?: Bilgisayarla "yapay veriler" üretebiliriz ama bu verilerin geçmişteki gerçek bilgiler kadar güvenilir olup olmadığı henüz bilinmiyor.**

- **Sonuç: Yapay zekâ bir sihirli değnek değil, doğru verilerle beslenmesi gereken bir araçtır.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

33- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: "Dijital İkiz" kavramı ile optimizasyon arasında nasıl bir bağ kurabiliriz?

- **Cevap: Dijital ikiz üzerinde binlerce farklı senaryo deneyerek (toptan deneme-yanılma), iklim krizini durdurmak için "en iyi politikayı" veya "en uygun çözümü" bulabiliriz (yani bir optimizasyon süreci işletebiliriz).**

Soru 2: Yapay zekâ neden kutup bölgeleri hakkında daha sağlıklı veriler oluşturmamıza yardım eder?

- **Açıklama/Cevap: Çünkü her santimetreyi izleyecek kadar çok insanımız yok. Yapay zekâ, eldeki az bilgiden yola çıkarak eksik parçaları tamamlayan modeller oluşturabilir.**

Soru 3: Bilim insanları neden "veriler yetersiz" diyor?

- **Cevap: Yapay zekâ algoritmaları, tıpkı bizim dersin başındaki "N sayıda etmen" gibi, ne kadar çok gerçek örnek görürse o kadar doğru tahmin yapar. Veri az olunca, algoritma yanlış yola sapabilir.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

6. İLAVE ETKİNLİK: Karınca Kurye İş Başında

- Problemimiz: Birbirine yollarla bağlı 5 tane şehrimiz var (Şehir 0, 1, 2, 3, 4).
- Hedef: Tüm şehirlere uğrayan ve en kısa sürede eve dönen rotayı bulmak.
- Zorluk: Şehirler arasındaki mesafeler birbirinden farklı!
- Çözüm: Karınca Koloni Optimizasyonu (KKO) algoritmasını kullanacağız.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Çizelge 7.1: Mesafeler Tablosu

- **Sayılar Ne Diyor?: Tablodaki satır ve sütunlar şehirleri temsil eder. Örneğin 0. satır ile 1. sütunun birleştiği yerdeki "2", Şehir 0 ile Şehir 1 arasındaki mesafedir.**
- **"inf" (Sonsuz) Nedir?: Bir şehirden kendisine gidemezsin! Bu yüzden aradaki mesafeyi "sonsuz" (inf) kabul ediyoruz ki karıncalar yanlışlıkla orada dönüp durmasın.**

Çizelge 7.1. Şehirlerarasındaki mesafe değerlerinin tanımı.

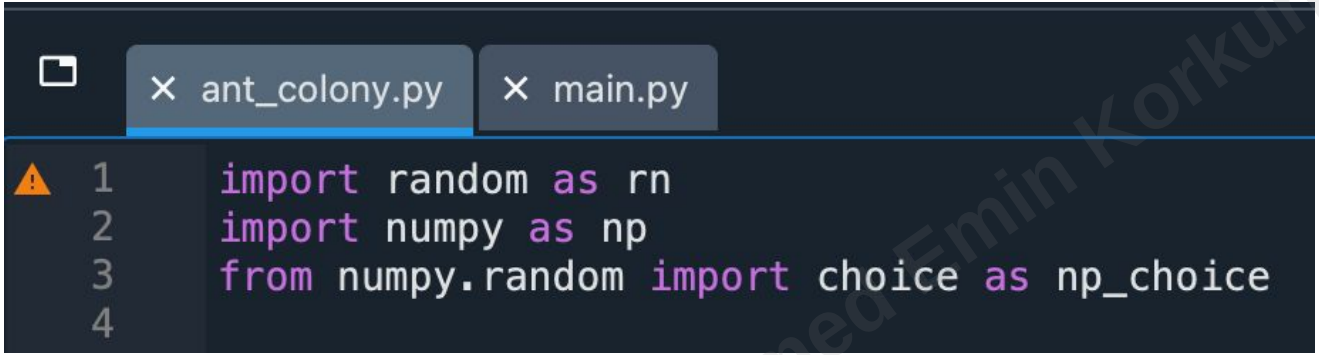
	0	1	2	3	4
0	inf	2	2	5	7
1	2	inf	4	8	2
2	2	4	inf	1	3
3	5	8	1	inf	2
4	7	2	3	2	inf

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1. Kütüphanelerin Tanımlanması



`import random as rn`: Python'un standart rastgele sayı üreticisini `rn` adıyla dahil eder (genel rastgelelik işlemleri için).
`import numpy as np`: Karmaşık matris işlemleri ve dizi (array) hesaplamaları için gelişmiş `numpy` kütüphanesini `np` adıyla dahil eder.
`from numpy.random import choice as np_choice`: Numpy içindeki rastgele seçim fonksiyonunu çağırır; karıncanın bir sonraki şehri belirli olasılıklara göre seçmesini sağlar.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

37- Karınca kolonisi parametreler

Karınca Kolonisi Sınıf Yapısı ve Parametre Tanımlamaları

```python
class AntColony(object):
    def __init__(self, distances, n_ants, n_best, n_iterations, decay, alpha=1, beta=1):
        self.distances  = distances
        self.pheromone = np.ones(self.distances.shape) / len(distances)
        self.all_inds = range(len(distances))
        self.n_ants = n_ants
        self.n_best = n_best
        self.n_iterations = n_iterations
        self.decay = decay
        self.alpha = alpha
        self.beta = beta
```

`class AntColony(object):`: Karınca Kolonisi Algoritması'nın tüm özelliklerini barındıran ana sınıfı tanımlar.
`def __init__(self, ...):`: Sınıf başlatıcı fonksiyonudur; algoritmanın çalışması için gerekli ayarları sisteme yükler.
`self.distances = distances`: Şehirler arasındaki uzaklık bilgilerini içeren matrisi hafızaya alır.
`self.pheromone = np.ones(...) / len(distances)`: Başlangıçta tüm yollara eşit ve düşük miktarda feromon (iz) serper.
`self.all_inds = range(len(distances))`: Gidilebilecek tüm şehirlerin indeks listesini (0, 1, 2...) oluşturur.
`self.n_ants = n_ants`: Her bir turda yola çıkarılacak toplam karınca sayısını belirler.
`self.n_best = n_best`: Tur sonunda sadece en iyi sonuca sahip kaç karıncanın yola feromon bırakacağını belirler.
`self.n_iterations = n_iterations`: Algoritmanın toplamda kaç döngü (nesil) boyunca çalışacağını ayarlar.
`self.decay = decay`: Feromonların zamanla uçup gitme (buharlaşma) oranını belirler (örn: 0.95).
`self.alpha = alpha`: Karıncanın kararlarında feromon izine (geçmiş tecrübeye) ne kadar önem vereceğini belirler.
`self.beta = beta`: Karıncanın kararlarında mesafeye (yakınlık/sezgisellik) ne kadar önem vereceğini belirler.

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

38- RUN

```
16
17      def run(self):
18          shortest_path = None
19          all_time_shortest_path = ("placeholder", np.inf)
20          for i in range(self.n_iterations):
21              all_paths = self.gen_all_paths()
22              self.spread_pheronome(all_paths, self.n_best, shortest_path=shortest_path)
23              shortest_path = min(all_paths, key=lambda x: x[1])
24
25              if shortest_path[1] < all_time_shortest_path[1]:
26                  all_time_shortest_path = shortest_path
27
28              # Feromon buharlaşması
29              self.pheromone = self.pheromone * self.decay
30          return all_time_shortest_path
```

Muhammed Emin Korkunç

`def run(self):`: Algoritmayı başlatan ve tüm iterasyonları yöneten ana fonksiyonu tanımlar.

`shortest_path = None`: O anki turda (iterasyonda) bulunan en kısa yolu tutmak için boş bir değişken açar.

`all_time_shortest_path = ("placeholder", np.inf)`: Tüm zamanların en iyi yolunu saklar; başlangıçta mesafeyi sonsuz (`inf`) kabul eder.

`for i in range(self.n_iterations):`: Belirlenen iterasyon sayısı kadar karıncaları tekrar tekrar yola çıkarır.

`all_paths = self.gen_all_paths()`: Tüm karıncalara rotalarını tamamlatır ve izledikleri yolları listeye kaydeder.

`self.spread_pheronome(...)`: Karıncaların bulduğu sonuçlara göre yollardaki feromon miktarlarını günceller.

`shortest_path = min(all_paths, key=lambda x: x[1])`: Mevcut tur içinde en kısa mesafeyi kat eden karıncanın yolunu bulur.

`if shortest_path[1] < all_time_shortest_path[1]:`: Eğer bu turda bulunan yol, şimdiye kadarki en iyi yoldan daha kısaysa rekoru günceller.

`self.pheromone = self.pheromone * self.decay`: Yollardaki feromonları buharlaştırarak eski ve zayıf izlerin silinmesini sağlar.

`return all_time_shortest_path`: Tüm döngüler bittiğinde algoritmanın bulduğu en ideal rotayı ve mesafeyi geri döndürür.

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

39- Feromon Güncelleme ve Yol Üretim Mantığı

```python
    def spread_pheronome(self, all_paths, n_best, shortest_path):
        sorted_paths = sorted(all_paths, key=lambda x: x[1])
        for path, dist in sorted_paths[:n_best]:
            for move in path:
                self.pheromone[move] += 1.0 / self.distances[move]

    def gen_path_dist(self, path):
        total_dist = 0
        for ele in path:
            total_dist += self.distances[ele]
        return total_dist

    def gen_all_paths(self):
        all_paths = []
        for i in range(self.n_ants):
            path = self.gen_path(0)
            all_paths.append((path, self.gen_path_dist(path)))
        return all_paths
```

`spread_pheronome` (Feromon Serpiştirme)

- **`sorted_paths = sorted(...)`**: Karıncaların bulduğu tüm yolları mesafelerine göre en kısadan en uzuna doğru sıralar.
- **`for path, dist in sorted_paths[:n_best]:`**: Sadece en iyi sonucu veren (`n_best` kadar) elit karıncaların geçtikleri yollara feromon eklemek için döngü başlatır.
- **`for move in path:`**: Elit karıncanın rotasındaki her bir adımı (şehirden şehre geçişi) tek tek inceler.
- **`self.pheromone[move] += 1.0 / self.distances[move]`**: Yol ne kadar kısaysa o yola o kadar çok feromon bırakır; böylece kısa yollar daha çekici hale gelir.

`gen_path_dist` (Yol Mesafesi Hesaplama)

- **`total_dist = 0`**: Bir karıncanın kat ettiği toplam mesafeyi toplamak için bir sayaç başlatır.
- **`for ele in path:`**: Karıncanın rotasındaki her bir şehir çiftini (kenarı) sırayla alır.
- **`total_dist += self.distances[ele]`**: Her adımın uzunluğunu ana mesafe tablosundan bakarak toplama ekler.

`gen_all_paths` (Tüm Karıncaları Yola Çıkarma)

- **`all_paths = []`**: Tüm karıncaların bulacağı rotaları ve mesafeleri saklamak için boş bir liste açar.
- **`for i in range(self.n_ants):`**: Kolonideki her bir karınca için sırayla işlem başlatır.
- **`path = self.gen_path(0)`**: Karıncayı 0 numaralı şehirden başlatarak tüm şehirleri dolaştırır.
- **`all_paths.append(...)`**: Karıncanın çizdiği rotayı ve hesaplanan toplam mesafesini listeye kaydeder.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

40- Karınca İçin Rota Oluşturma Mantığı

```python
    def gen_path(self, start):
        path = []
        visited = set()
        visited.add(start)
        prev = start
        for i in range(len(self.distances) - 1):
            move = self.pick_move(self.pheromone[prev], self.distances[prev], visited)
            path.append((prev, move))
            prev = move
            visited.add(move)
        path.append((prev, start))
        return path
```

`def gen_path(self, start):`: Karınca için belirli bir şehirden başlayarak tam bir rota çizen fonksiyonu tanımlar.
`path = []`: Karıncanın geçtiği yolları (şehir çiftlerini) saklamak için boş bir liste oluşturur.
`visited = set()`: Aynı şehre tekrar uğramamak için ziyaret edilen şehirlerin kaydını tutacak bir küme açar.
`visited.add(start)`: Başlangıç şehrini "ziyaret edildi" olarak işaretler.
`prev = start`: Karıncanın bulunduğu mevcut konumu, bir sonraki adıma referans olması için kaydeder.
`for i in range(len(self.distances) - 1):`: Toplam şehir sayısının bir eksiği kadar adım atarak tüm şehirlerin gezilmesini sağlar.
`move = self.pick_move(...)`: Mevcut feromon ve mesafe bilgilerini kullanarak karıncanın gideceği bir sonraki şehri seçer.
`path.append((prev, move))`: Gidilen yolu (nereden, nereye) karıncanın rotasına ekler.
`prev = move`: Gidilen yeni şehri, bir sonraki adımın başlangıç noktası yapar.
`visited.add(move)`: Yeni gidilen şehri "ziyaret edildi" listesine ekler.
`path.append((prev, start))`: Tüm şehirler bittiğinde, karıncanın turu tamamlaması için başladığı şehre dönüş yolunu ekler.
`return path`: Karıncanın oluşturduğu tam rotayı (şehirler arası geçişleri) geri döndürür.

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

41- Karıncanın Olasılıksal Seçim Mantığı

```
63
64      def pick_move(self, pheromone, dist, visited):
65          pheromone = np.copy(pheromone)
66          pheromone[list(visited)] = 0
67
68          # ACO Olasılık Formülü: (Feromon^alpha) * (Görünürlük^beta)
69          row = (pheromone ** self.alpha) * ((1.0 / dist) ** self.beta)
70
71          norm_row = row / row.sum()
72          move = np_choice(self.all_inds, 1, p=norm_row)[0]
73          return move
```

Muhammed Emin Korkunç

def pick_move(self, pheromone, dist, visited):: Karıncanın önündeki feromon, mesafe ve ziyaret geçmişine bakarak seçim yaptığı fonksiyonu tanımlar.

pheromone = np.copy(pheromone): Mevcut feromon değerlerini korumak ve üzerinde işlem yapabilmek için bir kopyasını oluşturur.

pheromone[list(visited)] = 0: Daha önce gidilmiş şehirlerin feromon değerini sıfırlar; böylece karıncanın aynı şehre tekrar gitmesi engellenir.

row = (pheromone ** self.alpha) * ((1.0 / dist) ** self.beta): ACO Formülü: Feromon yoğunluğunu (α) ve mesafenin tersini (β) çarparak her yol için bir "çekicilik" puanı hesaplar.

norm_row = row / row.sum(): Hesaplanan puanları toplam puana bölerek, hepsinin toplamının 1 (yani %100) olduğu bir olasılık dağılımına dönüştürür.

move = np_choice(self.all_inds, 1, p=norm_row)[0]: Oluşturulan olasılık tablosuna (p=norm_row) göre rastgele bir şehir seçer (yüksek olasılıklı şehirlerin seçilme şansı daha fazladır).

return move: Karıncanın gitmeye karar verdiği yeni şehrin numarasını döndürür.

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

42- Karıncalar Nasıl İlerler?

Şekil 7.11 & 7.13: Karar Verme Zamanı!

Kodun mutfağında iki önemli olay dönüyor:

1. **spread_pheromone (Koku Bırak): En kısa yolu bulan karıncanın geçtiği yollara daha fazla "puan" (feromon) eklenir.**
2. **pick_move (Şehir Seçimi): Karınca bir yol ayrımına geldiğinde; yolun hem kısa olmasına hem de üzerinde çok koku olmasına bakar.**
    - ***Zekice:* Daha önce gittiği şehre bir daha gitmemek için o şehri listesinden siler!**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

43- Karıncalar Nasıl İlerler?

Şekil 7.16: En Akıllı Rota Bulundu mu?

Kodumuz çalıştığında ekrana şampiyon rotayı yazdırır:

- **Rota Örneği: `(0, 2), (2, 3), (3, 4), (4, 1), (1, 0)`**
- **Anlamı: Şehir 0'dan 2'ye, oradan 3'e, sonra 4'e, sonra 1'e git ve en son başladığın yere (0'a) geri dön!**
- **Toplam Mesafe: 9.0 birim. Bilgisayarımız milyonlarca olasılık arasından en kısa olanı buldu!**

ant_colony.py | main.py

```python
import numpy as np
import os
import sys

# Dosyaların bulunduğu klasörü Python'a tanıtıyoruz
su_an_ki_dizin = os.path.dirname(os.path.abspath(__file__))
if su_an_ki_dizin not in sys.path:
    sys.path.append(su_an_ki_dizin)

from ant_colony import AntColony

# Mesafe Matrisi (Örnek Problem)
distances = np.array([[np.inf, 2, 2, 5, 7],
                      [2, np.inf, 4, 8, 2],
                      [2, 4, np.inf, 1, 3],
                      [5, 8, 1, np.inf, 2],
                      [7, 2, 3, 2, np.inf]])
```

```python

# Algoritmayı yapılandır
# (mesafeler, karınca, en iyiler, iterasyon, buharlaşma, alpha, beta)
ant_colony = AntColony(distances, 15, 3, 100, 0.95, alpha=1, beta=2)

# Çalıştır
shortest_path = ant_colony.run()

print("\n--- ÇALIŞMA TAMAMLANDI ---")
print("Bulunan En Kısa Rota: {}".format(shortest_path[0]))
print("Toplam Mesafe: {}".format(shortest_path[1]))
```

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

44- Yarış Başlasın ve Sonuçlar!

```
--- ÇALIŞMA TAMAMLANDI ---
Bulunan En Kısa Rota: [(0, 2), (2, 3), (3, 4), (4, 1), (1, 0)]
Toplam Mesafe: 9.0

In [71]:
```

Geçiş	Mesafe	Açıklama
(0, 2)	2	Başlangıçtan 2. şehre gidiş.
(2, 3)	1	2'den 3. şehre gidiş (En kısa bağlantılardan biri).
(3, 4)	2	3'den 4. şehre gidiş.
(4, 1)	2	4'den 1. şehre gidiş.
(1, 0)	2	1'den başlangıca (0) dönüş.
TOPLAM	**9.0**	

🐜 Karıncalar Neyi Başardı?

Karıncalar bu süreçte "Açgözlü" (Greedy) davranıp sadece o anki en kısa yolu seçselerdi bazen tıkanabilirlerdi. Ancak feromon birikimi sayesinde, bireysel karıncalar rastgele yollar denese de, kolonin toplam hafızası bu en ideal döngüyü (cycle) bulup sabitledi.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

45- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Kodda `n_ants` sayısını 10 yerine 100 yapsaydık ne değişirdi?

- **Cevap: Daha fazla karınca yollara düşerdi. Bu, en kısa yolu daha çabuk bulmamızı sağlayabilir ama bilgisayarımızı biraz daha fazla yorar.**

Soru 2: Bir yolu çok koku (feromon) varsa karınca orayı kesin mi seçer?

- **Açıklama/Cevap: Kesin değil, ama orayı seçme olasılığı (ihtimali) çok artar. `pick_move` fonksiyonu bu seçimi olasılık hesaplayarak yapar.**

Soru 3: Şehirler arasındaki mesafeleri değiştirdiğimizde kodumuz hâlâ çalışır mı?

- **Cevap: Elbette! `distances` matrisindeki sayıları değiştirdiğin anda karıncalar yeni haritaya göre en iyi yolu aramaya başlar. İşte zeki optimizasyonun gücü budur!**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Optimizasyon Dünyasında Verinin Rolü

- **Hangi Veri Setlerini Kullanabiliriz?**
  - **Cevap: Şehirler arası mesafe tabloları, fabrikadaki makine çalışma saatleri, ürün içerik oranları (kimyasal karışımlar) veya borsa gibi sayısal tahmin verileri en uygun örneklerdir.**
  - **Açıklama: Optimizasyon "en iyiyi" aradığı için, içinde sayılar ve seçenekler olan her türlü liste bizim için bir veri setidir.**
- **Veri Seti Başarıyı Nasıl Etkiler?**
  - **Cevap: Veri ne kadar karmaşıksa (değişken sayısı fazlaysa), pygad gibi kütüphanelerin sunduğu zeki algoritmalar klasik yöntemlere göre çok daha başarılı olur.**
  - **Açıklama: Çok değişkenli (yüksek dereceli) problemlerde klasik matematik tıkanırken, zeki optimizasyon "rastgele arama ve yardımlaşma" sayesinde çözüme ulaşır.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Başlık: Kaggle ve GitHub'da Neler Bulabiliriz?

- Hangi Veriler Bizimle Uyumlu?
  - Uyumlu: Lojistik rotaları (Kurye yolları), enerji tüketim verileri veya bütçe planlama tabloları zeki optimizasyon için mükemmeldir.
  - Uyumsuz: Sadece fotoğraf veya düz yazı (metin) içeren veri setleri, üzerinde matematiksel bir "en iyileme" hesabı yapılamayacağı için doğrudan bu teknikle uyumlu değildir.
- Yapay Zekâ Tekniği Uyumluluğu:
  - Cevap: Eğer problemde bir "hedef değer" (örneğin 44'e ulaşmak) veya "en kısa yol" (9 birim mesafe) arıyorsak, elimizdeki sayısal veri seti bu teknikle tam uyumludur.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

48- Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Bir veri setinde çok fazla yanlış veya eksik bilgi varsa optimizasyon nasıl etkilenir?

- **Cevap: Algoritma "en iyiyi" ararken yanlış verilere takılıp kalabilir. "Dünya'dan Haberler" kısmında gördüğümüz gibi; yeterli ve doğru veri yoksa yapay zekâ optimum (en iyi) sonuçları veremez.**

Soru 2: Kaggle'dan bir "Hava Durumu Tahmini" verisi indirdik. Bunu Karınca Kolonisi ile mi yoksa Genetik Algoritma ile mi çözmek daha mantıklıdır?

- **Açıklama/Cevap: Eğer hava sıcaklığı gibi sayısal bir değer tahmin edeceksek Genetik Algoritma (Sürekli Optimizasyon) daha uygundur. Eğer fırtınanın izleyeceği en kısa rotayı bulmak istiyorsak Karınca Kolonisi (Kombinasyonel) daha mantıklıdır.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc